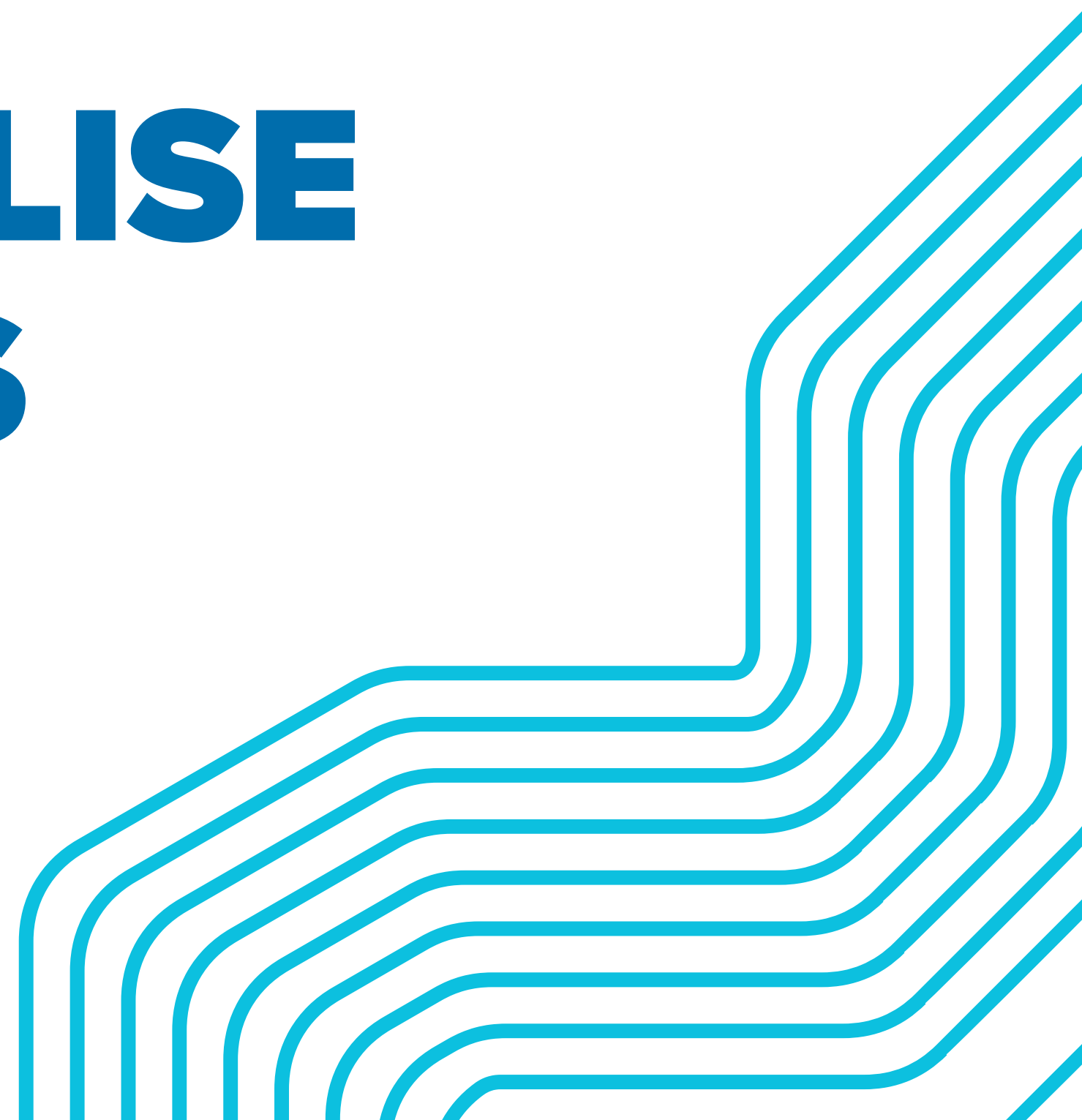




TRABALHO DE CRIAÇÃO E ANÁLISE DE ALGORITMOS

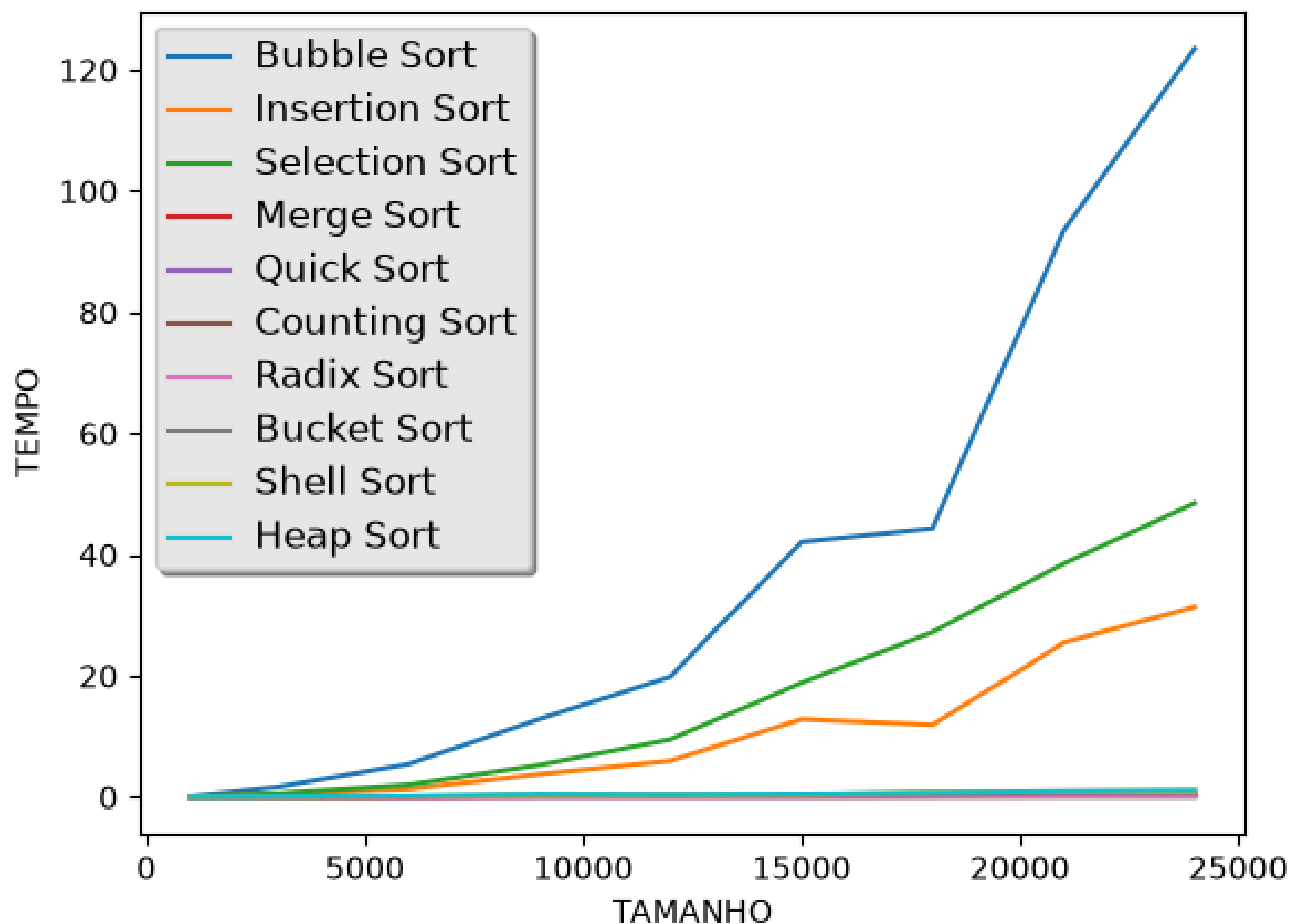
Equipe:
Alana Margarida
Antônio Vinicius
Gustavo Rodrigues
Iure Diniz



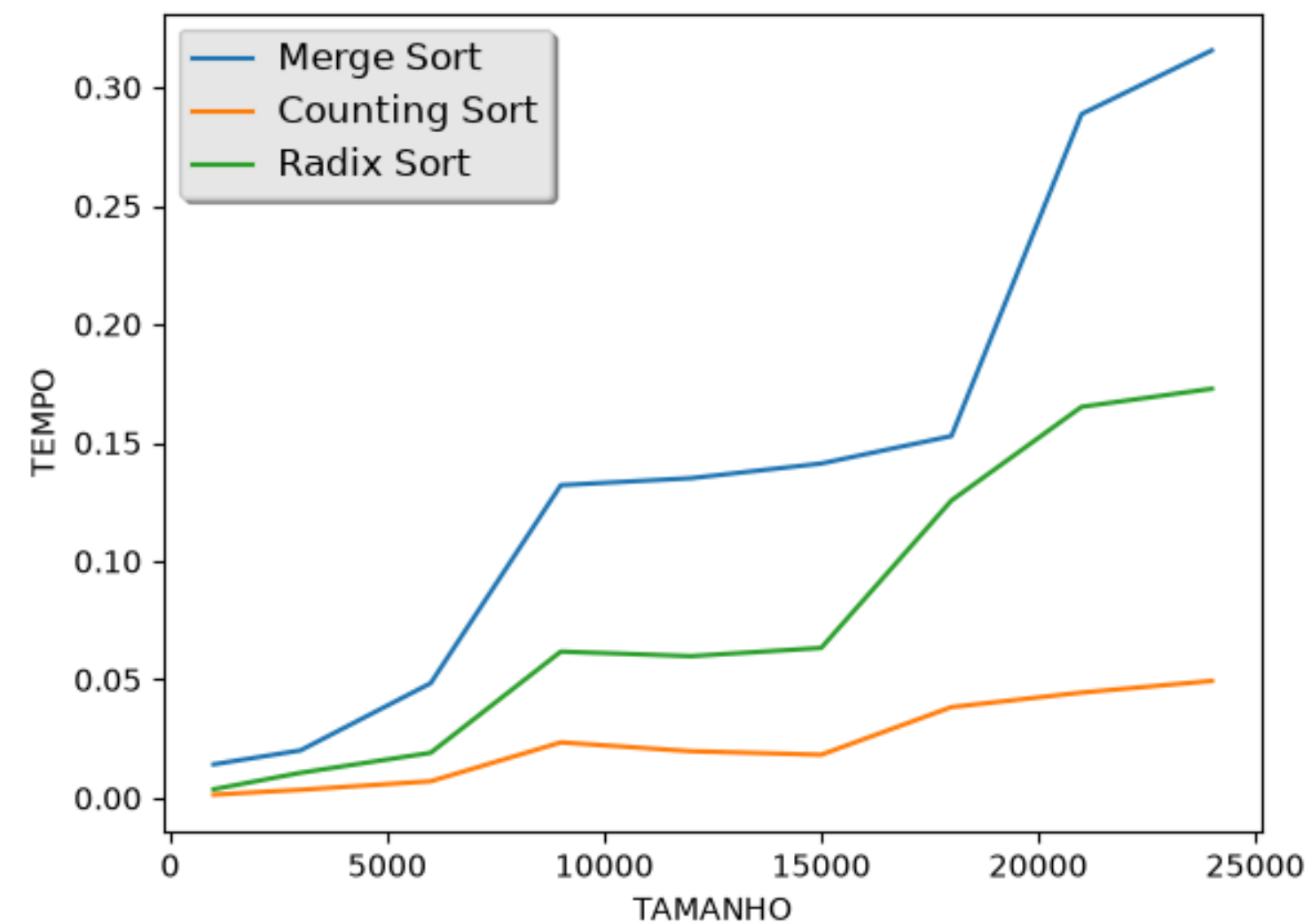
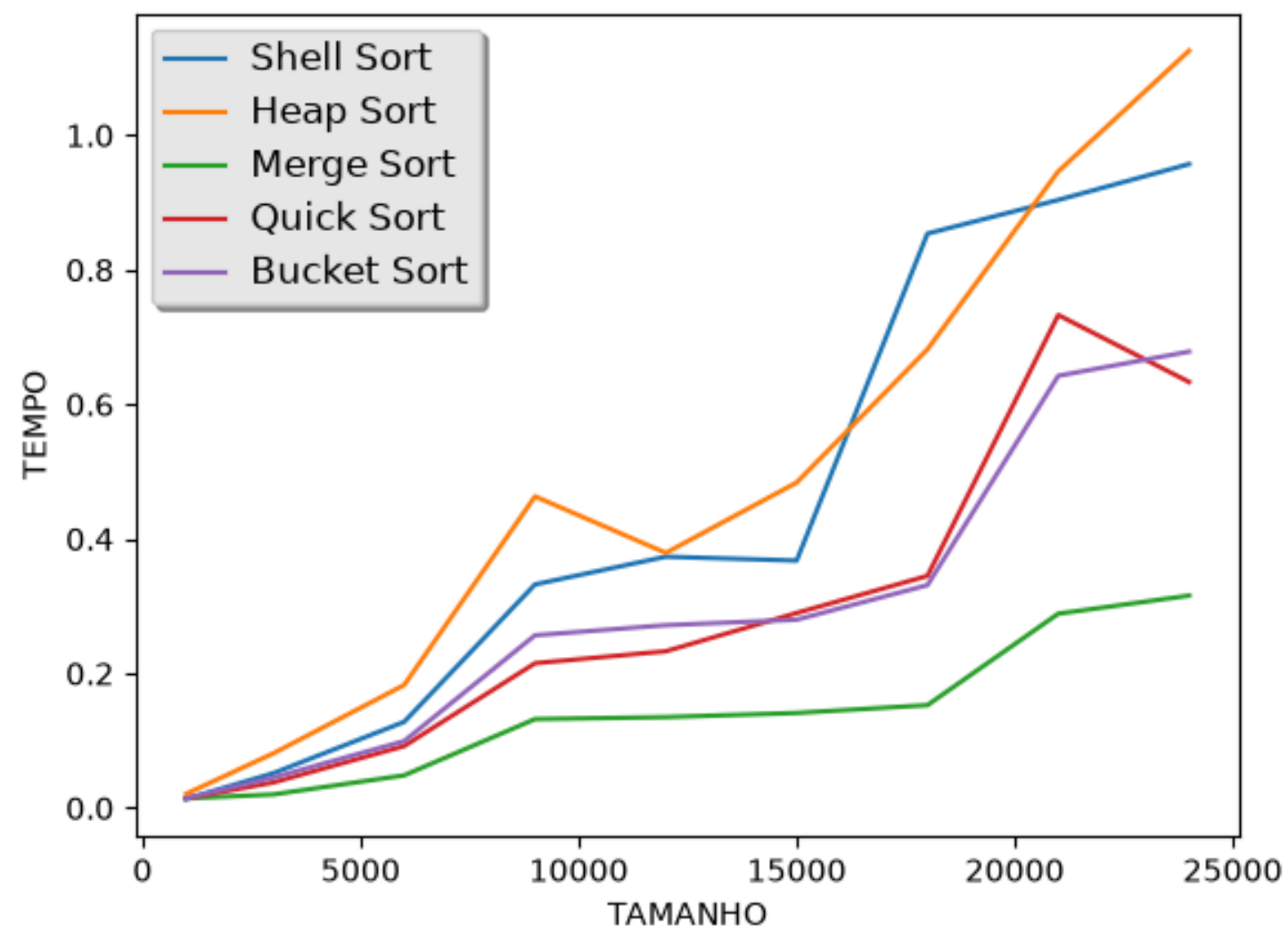
Questão A

A primeira questão do trabalho pede para que todos os 10 métodos de ordenação sejam implementados e comparados em gráficos criados pelo próprio programa em python.

Ao lado é possível ver o resultado obtido comparando todos os algoritmos de uma vez



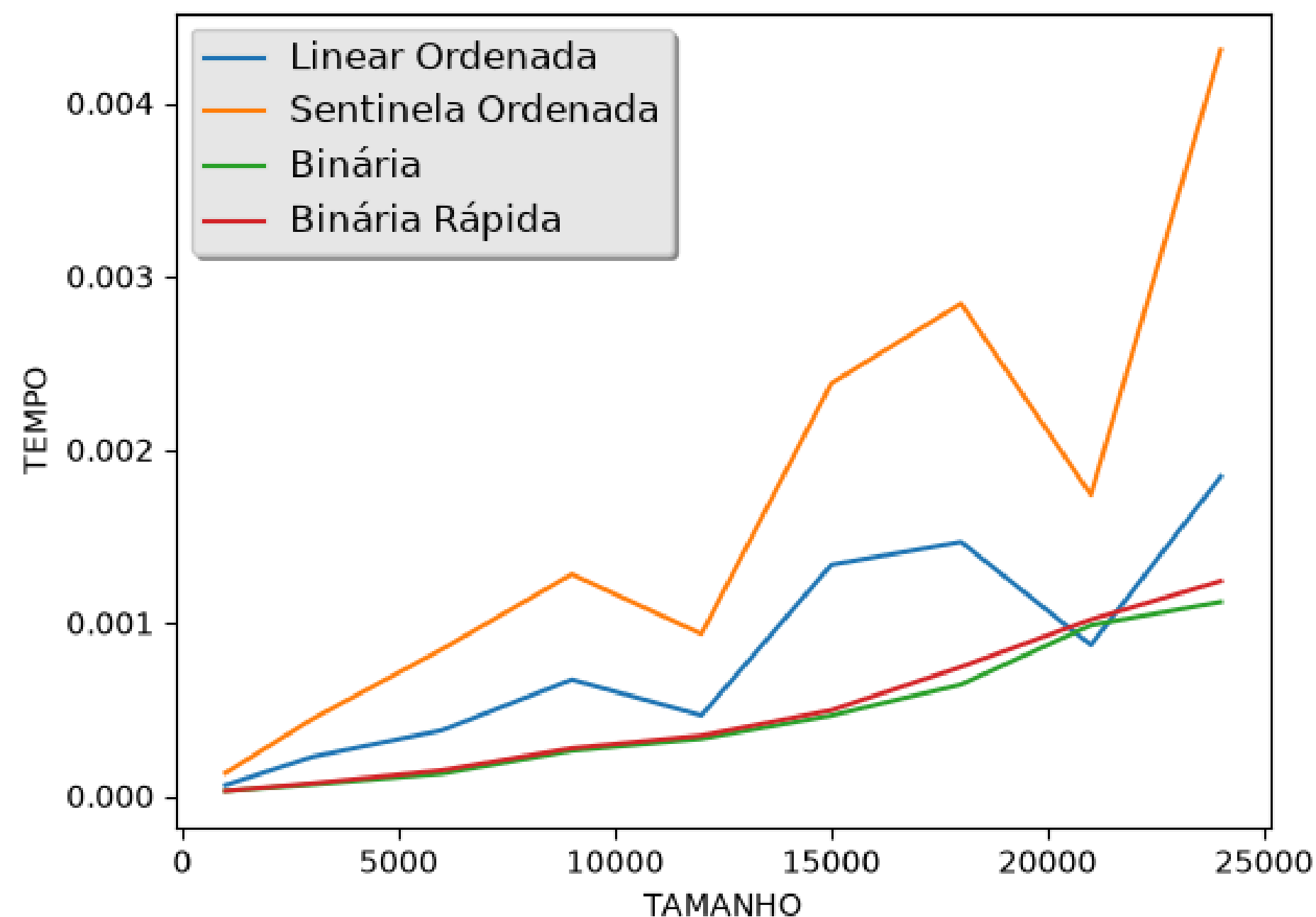
Questão A



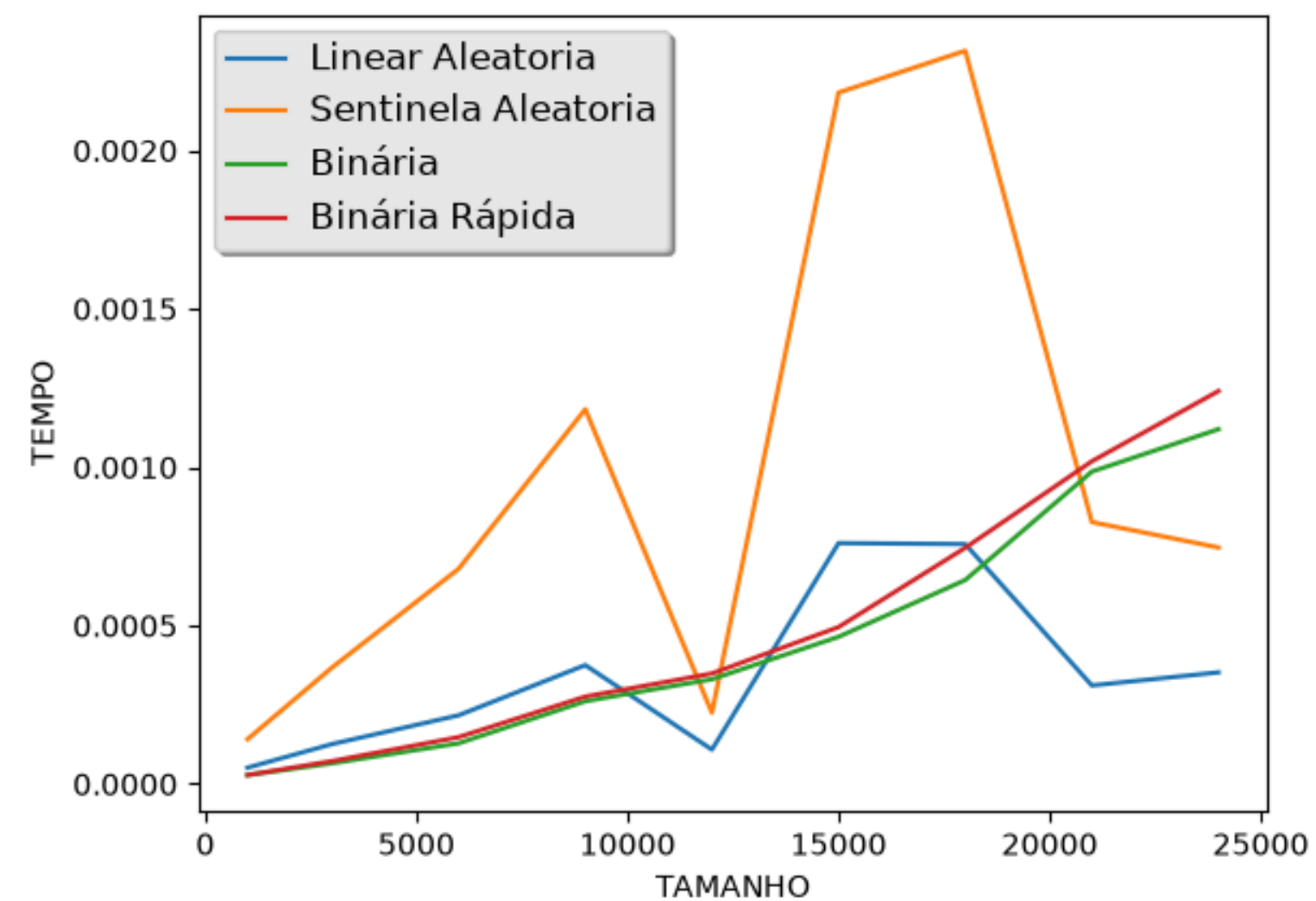
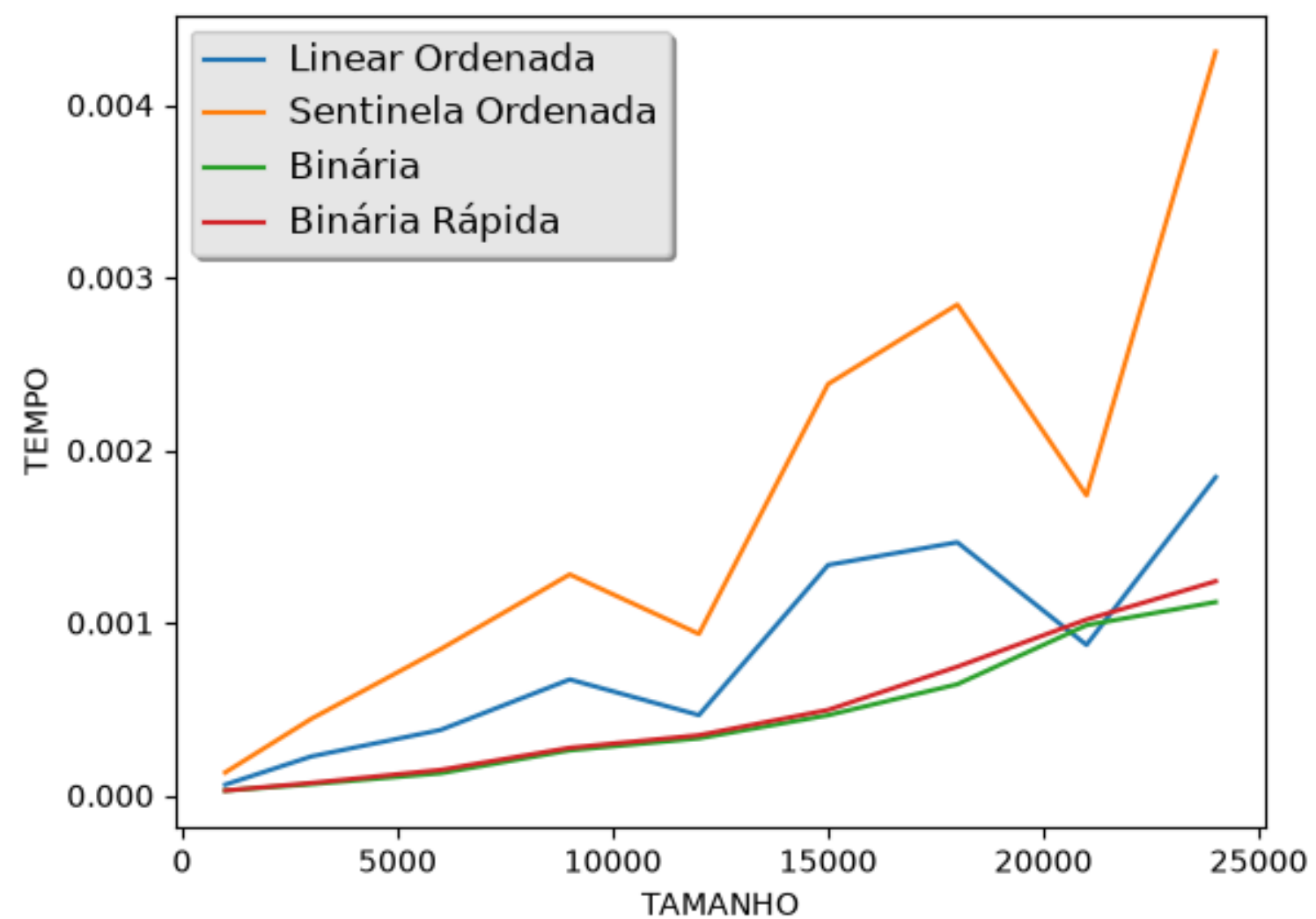
Questão B

A segunda questão geral pede para compararmos, agora, os seguintes algoritmos de busca:

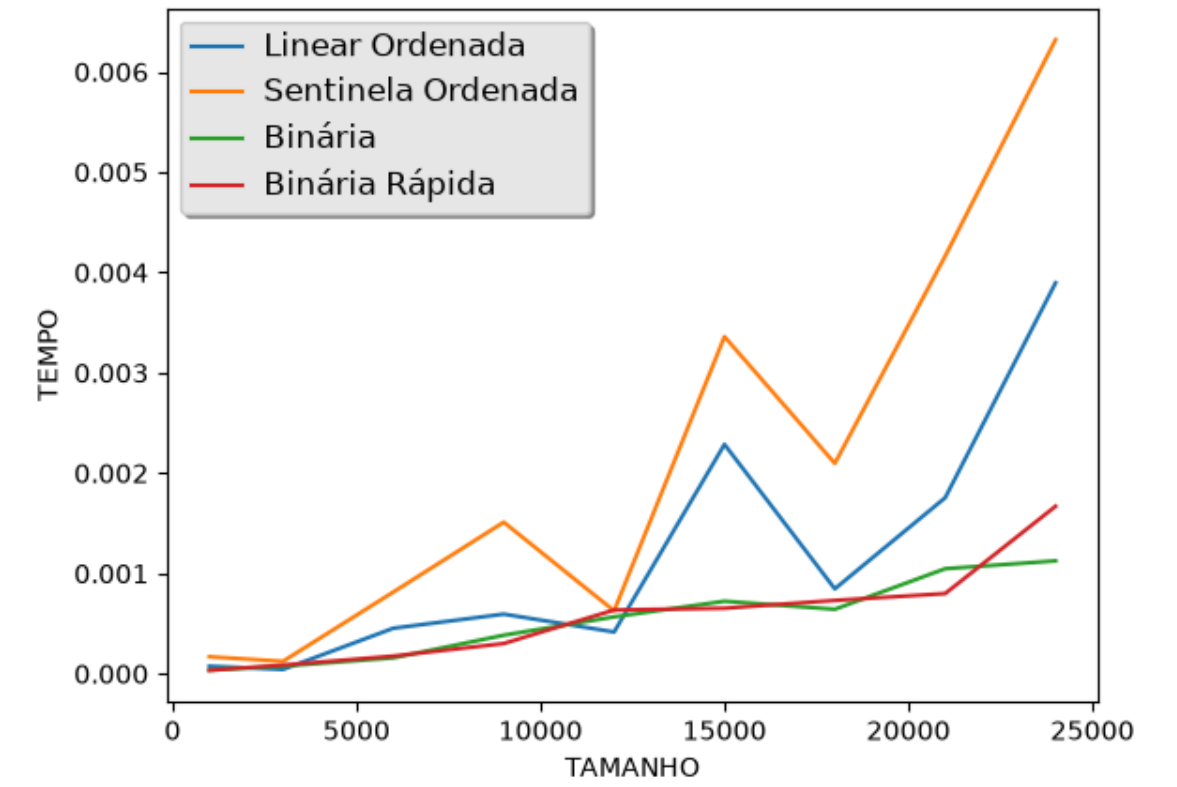
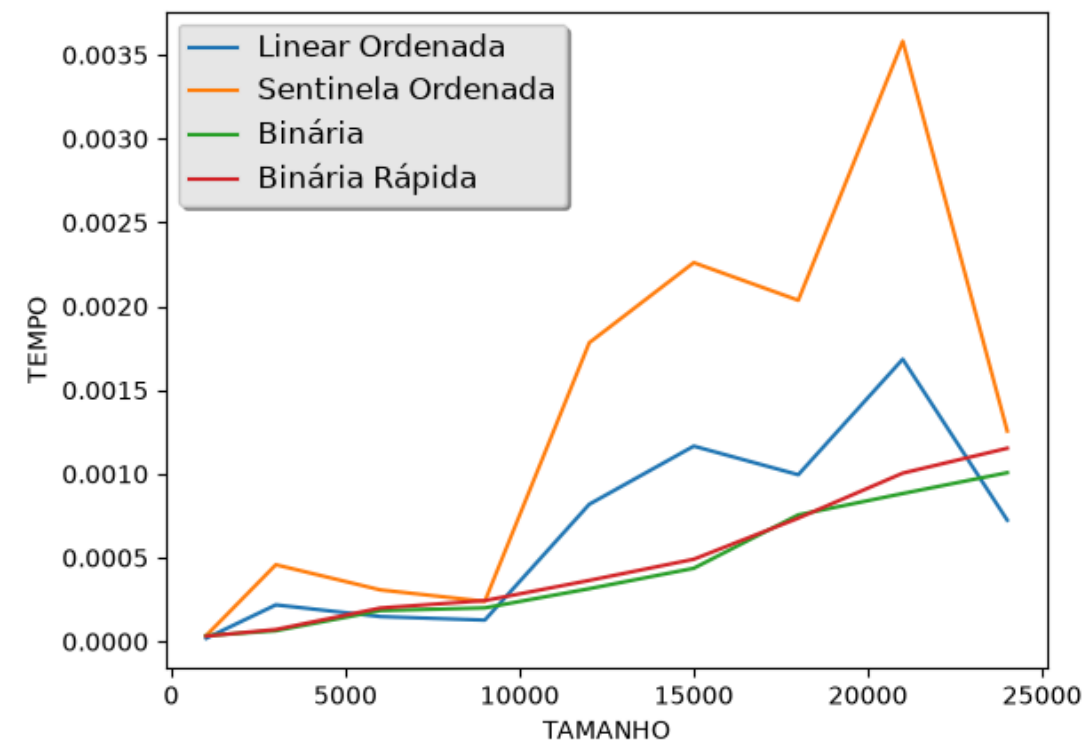
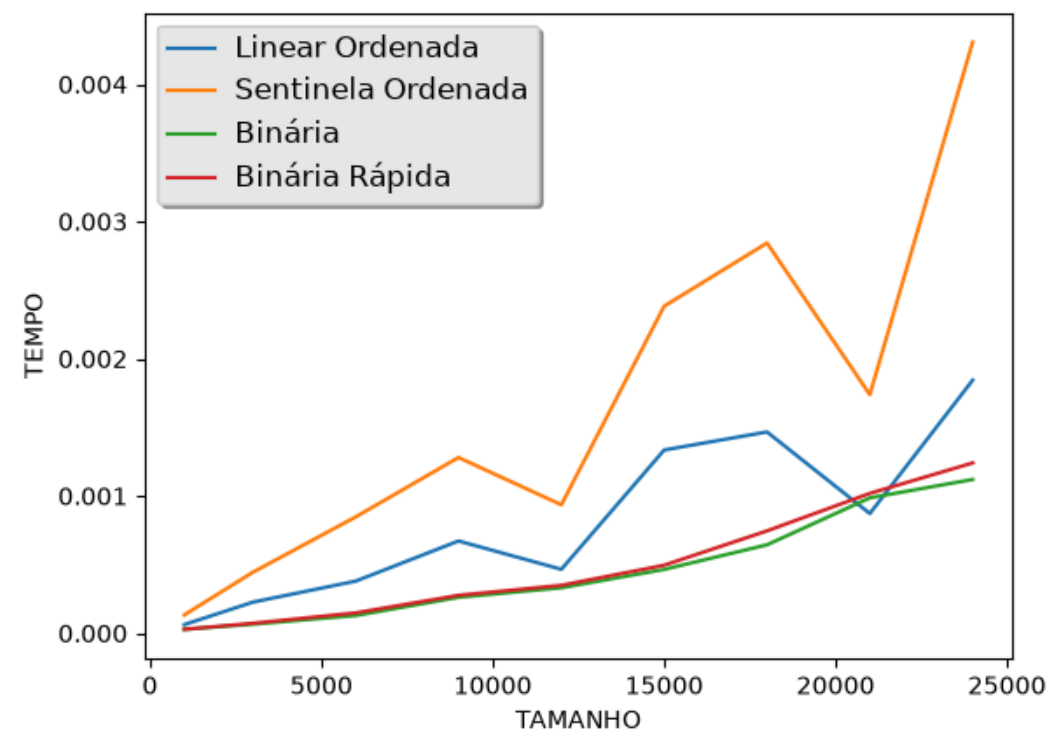
- Busca linear
- Busca linear com sentinela
- Busca binária
- Busca binária rápida



Questão B



Questão B



QUESTÃO 17



Crie um programa que dado uma string, coloque as letras dela em ordem crescente (Linguagem: Use C ou Python ou Ruby).



INICIALIZAÇÃO

```
1  texto = input("Digite uma string: ")
2
3  letras = list(texto)
4
5  for i in range(len(letras)):
6
7      if 'A' <= letras[i] <= 'Z':
8
9          codigo_ascii = ord(letras[i])
10         codigo_ascii = codigo_ascii + 32
11         letras[i] = chr(codigo_ascii)
12
```


SELECTION SORT

```
13  for i in range(len(letras) - 1):
14      menor = i
15
16      for j in range(i + 1, len(letras)):
17          if letras[j] < letras[menor]:
18              menor = j
19
20      temp = letras[i]
21      letras[i] = letras[menor]
22      letras[menor] = temp
```

```
25
26  resultado = ""
27
28  for letra in letras:
29      resultado += letra
30
31  print("String ordenada:", resultado)
```

RESULTADOS

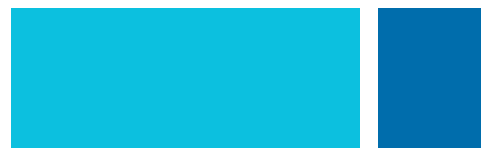
```
Digite uma string: ciencia da computação
String ordenada:  aaacccdeiimnooptuãç
○ PS C:\Users\pc\Desktop\CANA> |
```

```
Digite uma string: oi tudo bem?
String ordenada:  ?bdeimootu
○ PS C:\Users\pc\Desktop\CANA> |
```

QUESTÃO 18



Implementar o algoritmo Merge Sort para ordenar uma lista encadeada de números. Linguagens permitidas: C, C++, Java, Python e Ruby.



INICIALIZAÇÃO

"Representa um nó individual da lista encadeada"

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

"Estrutura da lista encadeada com funções auxiliares"

```
class LinkedList:
    def __init__(self):
        self.head = None

    # Adiciona um elemento ao final da lista
    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    # Imprime a lista de forma legível
    def print_list(self):
        temp = self.head
        elements = []
        while temp:
            elements.append(str(temp.data))
            temp = temp.next
        print(" -> ".join(elements) if elements else "Lista Vazia")
```

FUNÇÕES

"Função principal do Merge Sort para listas encadeadas"

```
def merge_sort(head):  
    # Se a lista estiver vazia ou tiver apenas um elemento  
    if not head or not head.next:  
        return head  
  
    # Encontra o meio da lista  
    middle = get_middle(head)  
    next_to_middle = middle.next  
  
    # Divide a lista em duas metades (quebra o vínculo)  
    middle.next = None  
  
    # Aplica o merge_sort recursivamente em ambas as metades  
    left = merge_sort(head)  
    right = merge_sort(next_to_middle)  
  
    # Intercala as duas metades ordenadas  
    sorted_list = sorted_merge(left, right)  
  
    return sorted_list
```

"Intercala duas listas encadeadas ordenadas em uma única lista ordenada"

```
def sorted_merge(left, right):  
    # Se uma das metades for vazia, retorna a outra  
    if not left:  
        return right  
    if not right:  
        return left  
  
    # Escolhe o menor valor e chama a função recursivamente para o próximo nó  
    if left.data <= right.data:  
        result = left  
        result.next = sorted_merge(left.next, right)  
    else:  
        result = right  
        result.next = sorted_merge(left, right.next)  
  
    return result
```


RESULTADO

Lista original:

38 -> 27 -> 43 -> 3 -> 9 -> 82 -> 10

Lista ordenada:

3 -> 9 -> 10 -> 27 -> 38 -> 43 -> 82

QUESTÃO 19



Implemente em Python os algoritmos de
busca sequencial e binária
para listas duplamente encadeadas



INICIALIZAÇÃO DA LISTA

```
def inicializar_lista(): 1 usage
    inicio = Item(0)
    tam = int(input("Digite o tamanho da lista: "))
    for i in range(1, tam+1):
        inserir_no_fim(inicio, i)
    return inicio
```

```
def inserir_no_fim(inicio, dado): 1 usage
    novoItem = Item(dado)
    if inicio is None:
        return novoItem

    atual = inicio
    while atual.prox:
        atual = atual.prox

    atual.prox = novoItem
    novoItem = atual
    return inicio
```


BUSCA LINEAR

```
def busca_linear(inicio, valor): 1 usage
    atual = inicio
    i = 0
    while atual:
        if atual.dado == valor:
            print(f"Iterações: {i}")
            return True
        else:
            atual = atual.prox
        i += 1
    print(f"Iterações: {i}")
    return False
```

BUSCA BINÁRIA

```
def encontrar_meio(inicio, fim): 1 usage
    if inicio is None:
        return None

    if inicio == fim:
        return inicio

    lento = inicio
    rapido = inicio.prox

    while rapido != fim:
        rapido = rapido.prox
        lento = lento.prox
        if rapido != fim:
            rapido = rapido.prox

    return lento
```

```
def busca_binaria(cabeca, valor): 1 usage
    comeco = cabeca
    fim = None
    i = 0
    while True:
        meio = encontrar_meio(comeco, fim)

        if meio is None:
            i += 1
            print(f"Iterações: {i}")
            return False

        if meio.dado == valor:
            i += 1
            print(f"Iterações: {i}")
            return True

        elif comeco == fim:
            break

        elif meio.dado < valor:
            comeco = meio.prox

        elif meio.dado > valor:
            fim = meio

    i+=1

    print(f"Iterações: {i}")
    return False
```

RESULTADOS

```
Digite o tamanho da lista: 40
Lista:
0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39
40 None
Digite um valor para ser buscado na lista: 25
Busca linear:
Iterações: 25
0 valor 25 foi encontrado
Busca binária:
Iterações: 5
0 valor 25 foi encontrado

Process finished with exit code 0
```

```
Digite o tamanho da lista: 30
Lista:
0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
30 None
Digite um valor para ser buscado na lista: 70
Busca linear:
Iterações: 31
0 valor 70 não foi encontrado
Busca binária:
Iterações: 6
0 valor 70 não foi encontrado

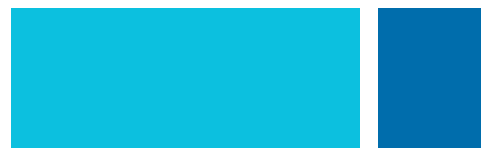
Process finished with exit code 0
```

QUESTÃO 20



Implemente em Java ou C++ ou Python os algoritmos recursivos e não-recursivos para determinar:

- a) O número de nós em uma árvore genérica.
- b) A soma dos conteúdos de todos os nós em uma árvore genérica, considerando que cada nó contém um inteiro.
- c) A profundidade de uma árvore genérica.



FUNÇÕES RECURSIVAS

```
def encontra(self, valor):  
    if self.valor == valor:  
        return self  
  
    for no in self.filho:  
        aux = no.encontra(valor)  
        if aux is not None:  
            return aux  
  
    return None
```

```
def contaNoRecursivo(self, n):  
    n += 1  
    for no in self.filho:  
        n = no.contaNoRecursivo(n)  
  
    return n  
  
def somaNoRecursivo(self, v):  
    v += self.valor  
    for no in self.filho:  
        v = no.somaNoRecursivo(v)  
  
    return v
```


FUNÇÕES RECURSIVAS

```
def subir(self, p):
    p += 1
    if self.pai is None:
        return p

    p = self.pai.subir(p)
    return p

def profundidadeNoRecursivo(self, valor, p):
    no = self.encontra(valor)

    if no is None:
        return p

    p = no.subir(p)

    return p
```

```
def profundidadeArvoreRecursivo(self):
    if len(self.filho) == 0:
        return 0

    maior = 0
    for no in self.filho:
        pFilho = no.profundidadeArvoreRecursivo()
        if pFilho > maior:
            maior = pFilho

    return 1 + maior
```

FUNÇÕES ITERATIVAS

```
def encontra(self, valor):  
    arvore = [self]  
  
    while arvore:  
        no = arvore.pop()  
  
        if no.valor == valor:  
            return no  
  
        arvore.extend(no.filho)  
  
    return None
```

```
def contaNo(self):  
    contador = 0  
    arvore = [self]  
  
    while arvore:  
        no = arvore.pop()  
        contador += 1  
        arvore.extend(no.filho)  
  
    return contador
```

```
def somaNo(self):  
    soma = 0  
    arvore = [self]  
  
    while arvore:  
        no = arvore.pop()  
        soma += no.valor  
        arvore.extend(no.filho)  
  
    return soma
```

FUNÇÕES ITERATIVAS

```
def profundidadeNo(self, valor):  
    no = self.encontra(valor)  
    p = -1  
  
    while no is not None:  
        p += 1  
        no = no.pai  
  
    return p
```

```
def profundidadeArvore(self):  
    p = 0  
    arvore = [self]  
  
    while arvore:  
        no = arvore.pop()  
        p = max(p, self.profundidadeNo(no.valor))  
        arvore.extend(no.filho)  
  
    return p
```


RESULTADOS

```
#####Criação da árvore#####
```

```
10  é a raiz da árvore
```

```
10  é pai de  5
```

```
10  é pai de  6
```

```
5   é pai de  15
```

```
6   é pai de  6
```

```
6   é pai de  7
```

```
6   é pai de  8
```

```
8   é pai de  9
```

```
#####Funções recursivas#####
```

```
0 numero de nós é:  8
```

```
A soma de todos os nós é:  66
```

```
A profundidade de 9 é: 3
```

```
A profundidade da árvore é: 3
```

```
#####Funções iterativas#####
```

```
0 numero de nós é:  8
```

```
A soma de todos os nós é:  66
```

```
A profundidade de 9 é: 3
```

```
A profundidade da árvore é: 3
```

REFERÊNCIAS

GueksforGueks.org

